KABARAK UNIVERSITY

School of Science, Engineering & Technology
Unit: Artificial Intelligence — COMP 413

CONTENT-BASED MOVIE RECOMMENDATION SYSTEM

Multi-Signal TF-IDF Similarity with Sparse Vector Architecture

Report Type	AI Mini-Project Group Report
Dataset	TMDB + IMDB Movies Dataset (Kaggle)
Team:	1. KALUME ALEX AMANI - INTE/MG/3261/09/21 2. ISAAC OTIENO - INTE/MG/3432/09/22 3. MARY CHELANGAT - INTE/MG/2073/09/22 4. PAUL KARUBI - INTE/MG/2724/09/22
Date:	February 2026

1. Problem Definition & Objective

Problem Statement

Every day, millions of people open streaming platforms and spend a frustrating amount of time scrolling through hundreds of movie titles without knowing what to watch.

The problem is not a lack of movies — it is the absence of a reliable, personalized guide to help each viewer find films that match their individual taste. Without a smart recommendation tool, viewers rely on generic popularity lists that ignore personal preferences, leading to poor user experience and missed discoveries.

Project Objective

To build a content-based movie recommendation system that accepts any movie title as input and returns a ranked list of the ten most similar movies. Similarity is determined by combining five complementary content signals extracted from rich movie metadata: genre labels, plot keywords, creative personnel (directors, writers, cast), plot overview text, and numeric quality metrics. These five signals are fused into a single weighted sparse feature matrix, enabling memory-efficient similarity computation at query time without ever materialising a full pairwise distance matrix.

Why Content-Based Filtering

The TMDB/IMDB dataset provides per-movie metadata rather than per-user rating logs. This makes collaborative filtering impossible, as there is no user behavior records to model. Instead, we extract and combine five expressive content signals that together describe the complete identity of a film:

- Genre labels — the category of film (Action, Drama, Science Fiction, etc.)
- Plot keywords — specific thematic and narrative elements (heist, dream, dystopia, wormhole)
- Creative personnel — directors (triple-weighted), writers, and top five cast members

- Overview text — the full plot synopsis, vectorized with bigrams to capture compound phrases
- Numeric metrics — vote average, vote count, revenue, runtime, and popularity, log-scaled and normalized

This is a pure content-based approach: similarity is derived entirely from what a movie is, not from how users have behaved. It is distinct from collaborative filtering (which requires user rating logs) and from hybrid systems (which combine both). The richness of five signals compensates for the absence of user behavior data.

Relevance & Impact

Recommendation systems are among the most commercially valuable AI applications in the world.

Netflix states its recommendation engine drives over 80% of content watched on the platform, saving the company more than one billion dollars (KES 129 billion) annually.

Amazon reports that 35% of its total revenue is generated directly through recommendations. This project simulates a real-world content-based pipeline at academic scale, using a genuine dataset of 367,909 movies with rich per-film metadata.

2. Data Acquisition

Data Source

The dataset used in this project is the TMDB + IMDB Movies Dataset published on Kaggle. It is a large-scale collection of movie metadata combining TMDB (The Movie Database) attributes with IMDB rating data, making it well suited for a rich content-based recommendation system.

Dataset URL: https://www.kaggle.com/datasets/ggtejas/tmdb-imdb-merged-movies-dataset

Dataset Overview

Attribute	Details
Total Records	367,909 movie records
Working Set	Top 50,000 movies filtered by vote count
File Format	CSV (.csv) — TMDB IMDB Movies Dataset.csv
Columns Used	title, genres, keywords, directors, writers, cast, overview, vote_average, vote_count, revenue, runtime, release_date, poster_path, popularity
Filtering Rationale	317,909 obscure films (near-zero votes) removed; only top 50K retained for meaningful similarity

Why This Dataset Suits a Content-Based System

- The genres column provides the content guardrail, ensuring thematically relevant recommendations.
- The keywords column encodes plot-level specificity that genre alone misses — two Action films can differ completely in theme, one a heist and one a war epic.

- The directors, writers, and cast columns encode creative identity — films by the same director or starring the same actor are meaningfully related regardless of genre.
- The overview column gives the full plot synopsis, enabling NLP-based thematic similarity using bigram TF-IDF.
- The vote_average, vote_count, revenue, runtime, and popularity columns encode quality and cultural impact signals that correlate with audience reception.
- Critically, the dataset contains no per-user rating rows. Collaborative filtering is therefore not applicable. The five content signals above substitute for user behavior data.

3. Data Cleaning & Preprocessing

This section covers every cleaning and preprocessing step applied to the raw dataset before feature engineering.

3.1 Load and Inspect

```
csv_files = [f for f in os.listdir(DATA_DIR) if
f.endswith('.csv')]
df = pd.read_csv(os.path.join(DATA_DIR, csv_files[0]),
low_memory=False)
df.columns = [c.strip().lower() for c in df.columns]
print(f'Raw rows: {len(df):,}')  # 367,909
```

3.2 Filter to Released Films Only

The dataset includes movies with various status values (Rumoured, In Production, Post Production). Only Released films are meaningful for recommendations, as unreleased films have no audience history.

```
df = df[df['status'] == 'Released'].copy()
```

3.3 Remove Nulls and Duplicates

```
df.dropna(subset=['title'], inplace=True)
df.drop_duplicates(subset=['title'], keep='first',
inplace=True)
```

3.4 Filter to Top 50,000 by Vote Count

Of the 367,909 raw movies, the vast majority have zero or near-zero votes and would produce meaningless similarity scores. We retain only the 50,000 most-voted movies, which represent the practically relevant catalogue. This step is critical not only for quality but for memory management.

```
vc_col = next((c for c in ['vote_count','numvotes'] if c
in df.columns), None)
df[vc_col] = pd.to_numeric(df[vc_col],
errors='coerce').fillna(0)
df = df.nlargest(50_000, vc_col)
```

3.5 Fill Missing Text and Numeric Fields

```
TEXT_COLS =
['genres','keywords','directors','writers','cast','overvi
ew']
for col in TEXT_COLS:
    df[col] = df[col].fillna('')


NUM_COLS =
['vote_average','vote_count','revenue','runtime','popular
ity']
for col in NUM_COLS:
    df[col] = pd.to_numeric(df[col],
errors='coerce').fillna(0).astype('float32')
```

3.6 Director Triple-Weighting

The director's creative identity is the single strongest discriminating content signal. Within the people corpus, each director token is inserted three times to give it proportionally higher TF-IDF weight relative to cast and writers.

```
def people_str(row):
    parts = []
    if row['directors']:
        d = clean(row['directors']).replace(' ', '_')
        parts += [d, d, d]          # triple weight for
director
    if row['writers']:
        parts.append(clean(row['writers']).replace(' ',
'_'))
    if row['cast']:
        parts += [n.strip().replace(' ', '_')
                  for n in
str(row['cast']).split(',')[:5]]
    return ' '.join(parts)
```

4. Exploratory Data Analysis

Exploratory analysis was conducted on the filtered 50,000-movie working set to understand the distribution of key features and validate design decisions in feature engineering.

4.1 Vote Count Distribution

The vote count distribution is heavily right-skewed. A small number of blockbusters have hundreds of thousands of votes while most films have a few hundred. This is the primary motivation for applying log-scaling (log1p) to vote_count, revenue, and popularity before normalisation, ensuring that blockbusters do not dominate the numeric similarity signal.

4.2 Genre Distribution

Drama and Comedy are the most common primary genres, each appearing in over 20% of the working set. The diversity across 19 distinct genre labels confirms that TF-IDF vectorization of the genre's column produces meaningfully different vectors for different movies, making the content signal discriminating.

4.3 Keyword Coverage

The keywords field has a median of approximately 8 keywords per movie, with popular films having up to 50. Keywords such as heist, dream, dystopia, wormhole, and superhero carry thematic specificity far beyond genre labels. The bigram TF-IDF applied to keywords captures compound concepts such as based on comic and artificial intelligence as single tokens, preserving their semantic integrity.

4.4 Director Representation

The top ten directors by film count in the dataset account for approximately 4% of the 50K working set. Directors with large filmographies create tight clusters in the people similarity space, validating the decision to triple-weight director tokens.

4.5 Dataset Summary Statistics

Metric	Value
Raw movie records	367,909
Working set	50,000 (top by vote count)
Distinct genre labels	~19
Average keywords per movie	~8
Most common genre	Drama (~22%)
Vote average range	0.0 – 10.0 (TMDB scale)
Combined feature matrix	50,000 × ~13,300 sparse (approx. 150–200 MB)

5. Feature Engineering

Feature engineering is the core intellectual contribution of this project. The central design principle is to store normalised feature vectors rather than materialise a pairwise similarity matrix. This keeps peak RAM under 4 GB regardless of dataset size.

The Memory Problem and Its Solution

A naive implementation would compute cosine_similarity(matrix) for each of the five signals, producing five N x N matrices. For N = 50,000 movies and five signals in float32:

5 × 50,000 × 50,000 × 4 bytes = 50 GB — not feasible in any Colab environment.

The solution is mathematically equivalent but architecturally different: instead of storing scores, we store the vectors. Similarity between any two movies is then a single dot product computed at query time. For L2-normalised vectors, dot product equals cosine similarity exactly.

5.1 Signal 1: Genre (TF-IDF)

Genre labels such as Action, Science Fiction, and Drama are pipe-separated in the raw data. After converting to space-separated strings and applying TF-IDF, rare genre combinations receive higher IDF weights than common ones such as Drama, making them more discriminating.

```
tfidf_genre = TfidfVectorizer(stop_words='english',
max_features=300)
M_genre =
normalize(tfidf_genre.fit_transform(genre_corpus).astype(
'float32'))
```

5.2 Signal 2: Keywords (Bigram TF-IDF)

Plot keywords are the most thematically specific signal. Using bigrams (ngram_range=(1,2)) captures compound concepts such as based on comic, artificial intelligence, and post apocalyptic as single tokens rather than splitting them into less meaningful unigrams.

```
tfidf_kw = TfidfVectorizer(stop_words='english',
max_features=2000, ngram_range=(1,2))
M_kw =
normalize(tfidf_kw.fit_transform(kw_corpus).astype('float
32'))
```

5.3 Signal 3: Creative Personnel (Triple-Weighted Directors)

The creative identity of a film is most strongly defined by its director. Two Christopher Nolan films share a cognitive style, visual grammar, and thematic preoccupations that no genre or keyword signal can fully capture. By inserting the director token three times in the people corpus, we give it proportionally higher TF-IDF weight relative to cast and writers.

```
# Director appears 3x, writers 1x, top-5 cast 1x each
# Names are de-spaced: 'Christopher Nolan' ->
'Christopher_Nolan'
tfidf_people = TfidfVectorizer(stop_words='english',
max_features=3000)
M_people =
normalize(tfidf_people.fit_transform(people_corpus).astyp
e('float32'))
```

5.4 Signal 4: Overview Text (Bigram TF-IDF)

The full plot synopsis is vectorised with bigram TF-IDF. This captures narrative structure at a thematic level — films about corporate espionage will share terms across their overviews, linking them even if their genre labels differ.

```
tfidf_ov = TfidfVectorizer(stop_words='english',
max_features=3000, ngram_range=(1,2))
M_ov =
normalize(tfidf_ov.fit_transform(overview_corpus).astype(
'float32'))
```

5.5 Signal 5: Numeric Quality Metrics

Five numeric dimensions are included: vote_average, vote_count, revenue, runtime, and popularity. Skewed columns are log-transformed with log1p before MinMax scaling to prevent blockbusters from dominating the signal.

```
for col in ['vote_count', 'revenue', 'numvotes',
'popularity']:
    num_arr[:, i] = np.log1p(num_arr[:, i])
num_arr =
normalize(MinMaxScaler().fit_transform(num_arr).astype('f
loat32'))
```

5.6 Weighted Combination into One Sparse Matrix

The five signal matrices are stacked horizontally with their weights applied, then the combined matrix is re-normalised. A single dot product against any row then produces the full weighted similarity score.

Signal	Weight	Justification
Genre (TF-IDF)	0.25	Content guardrail ensuring thematic relevance
Keywords (Bigram TF-IDF)	0.20	Plot-level specificity beyond genre labels
People (Directors ×3, Cast)	0.20	Creative identity — strongest discriminating signal
Overview (Bigram TF-IDF)	0.15	Thematic NLP similarity from plot synopsis
Numeric Metrics	0.20	Quality and cultural impact signal

```
combined = sp.hstack([
    M_genre   * 0.25,
    M_kw      * 0.20,
    M_people  * 0.20,
    M_ov      * 0.15,
    M_num     * 0.20,
], format='csr').astype('float32')
combined = normalize(combined)   # L2-normalise all rows
```

After normalisation, combined has shape (50,000 × ~13,300) and occupies approximately 150–200 MB as a sparse matrix — compared to the 50 GB five dense pairwise matrices would require.

6. Model Building

The system has three components: the offline training phase in Google Colab, the artefact storage layer as serialized PKL files, and the online inference phase in the Flask web application.

6.1 Offline Training Phase (Google Colab)

Training runs once in Google Colab and produces four serialised artefacts. The combined sparse matrix is the central model object.

```
pickle.dump(combined,    open('combined_matrix.pkl',
'wb'), protocol=4)
pickle.dump(movie_meta,  open('movie_meta.pkl',
'wb'), protocol=4)
pickle.dump(title_index, open('title_index.pkl',
'wb'), protocol=4)
pickle.dump(titles_list, open('titles_list.pkl',
'wb'), protocol=4)
```

6.2 Query-Time Inference

At query time, finding the top-N recommendations for any movie requires a single sparse matrix-vector multiplication. This is the key architectural advantage over pre-computing a full similarity matrix.

```
idx       = title_index[movie_title]
query_vec = combined[idx]                          #
sparse row vector
scores    = (combined @ query_vec.T).toarray().flatten()
# cosine sims
scores[idx] = -1                                   #
exclude self
top_idx   = np.argpartition(scores, -n)[-n:]       # O(N)
partial sort
top_idx   = top_idx[np.argsort(scores[top_idx])[::-1]]
```

Because every row of combined is L2-normalised, the dot product produces exact cosine similarity scores for all 50,000 movies in a single vectorised operation. Using np.argpartition reduces the ranking step from O(N log N) to O(N).

6.3 Sample Output — Inception (2010)

#	Movie	Score	Genre	Director
1	Interstellar (2014)	0.8821	Adventure, Sci-Fi	Christopher Nolan
2	The Dark Knight (2008)	0.8134	Action, Crime	Christopher Nolan
3	Memento (2000)	0.7943	Mystery, Thriller	Christopher Nolan
4	The Matrix (1999)	0.7612	Action, Sci-Fi	The Wachowskis
5	Fight Club (1999)	0.7201	Drama	David Fincher

Ranks 1–3 is driven strongly by the people signal (same director). Rank 4 (The Matrix) is driven by shared keywords and genre. Rank 5 (Fight Club) is driven by overview similarity and a comparable numeric profile despite having a different genre label.

7. Model Evaluation

Because this dataset has no per-user rating logs, quantitative Precision@K evaluation is not applicable. Evaluation is conducted qualitatively through signal ablation and thematic coherence inspection.

7.1 Signal Ablation — Qualitative Comparison

Input Movie	Genre Only	People Only	Full System (5 Signals)
Inception (2010)	Interstellar, Avatar, Avengers	The Dark Knight, Memento, Tenet	Interstellar, The Dark Knight, Memento
The Godfather (1972)	Scarface, Goodfellas, Heat	The Godfather Part II, Apocalypse Now	The Godfather Part II, Goodfellas, Scarface
Toy Story (1995)	Shrek, Ice Age, Kung Fu Panda	Toy Story 2, Toy Story 3, A Bug's Life	Toy Story 2, A Bug's Life, Toy Story 3

Genre-only over-broadens: Avatar and The Avengers are thematically distant from Inception despite being Sci-Fi. People-only is precise for directors with large filmographies but underserves cross-director thematic similarity. The full five-signal system surfaces recommendations that are simultaneously thematically relevant, directionally consistent, and numerically comparable in scale and audience reception.

7.2 Memory Efficiency Validation

Approach	RAM (50K movies)	RAM (367K movies)
Dense pairwise matrix (float64)	~20 GB	~1,080 GB — impossible
Dense pairwise matrix (float32)	~10 GB	~540 GB — impossible
Sparse combined feature matrix	**~150–200 MB**	**~1.5 GB (tractable)**

The sparse architecture is not a compromise. It is the correct engineering choice for recommendation systems at this scale. The mathematical output (cosine similarity scores) is identical to the dense approach; only the order of computation changes.

8. Results Interpretation & Insights

8.1 Key Findings

- The five-signal system produces recommendations that are thematically coherent, directionally consistent, and validated through numeric quality signals — a significant improvement over single-signal or two-signal designs.
- Director triple-weighting is the most impactful single design decision. Films by auteur directors (Nolan, Kubrick, Fincher, Kurosawa) cluster together strongly, which aligns with how audiences and critics actually organise films.
- Bigram TF-IDF on keywords outperforms unigram TF-IDF because compound concepts such as based on comic, time travel, and artificial intelligence are semantically single units. Splitting them loses meaning.
- Log-scaling numeric columns (vote_count, revenue, popularity) is essential. Without it, a film with 2 million votes would dominate similarity scores regardless of its actual relevance to the query film.
- The 50K working set filter removes 317K obscure titles with near-zero votes that produce meaningless similarity scores and waste computation. This is the most impactful preprocessing decision for practical recommendation quality.

8.2 Real-World Interpretation

In practical terms, the system behaves as a film critic who knows the full creative context of a movie — what it is about, who made it, what themes it explores, how audiences received it, and where it sits in the cultural landscape. When queried with The Godfather, the system recommends The Godfather Part II not merely because it shares a genre, but because it shares a director (Coppola), a writer (Puzo), multiple cast members, overlapping keywords (mafia, family, crime, loyalty), a similar overview theme, and a comparable audience reception profile.

8.3 Limitations

- No user personalisation: The system is item-to-item only. It finds movies similar to a given movie, not movies tailored to a specific user's watch history. Personalisation would require a separate collaborative component and user behaviour data.
- Cold-start for new releases: A film with no votes yet has a zero numeric signal vector, reducing its similarity to films with strong numeric profiles. The text signals still function but the combined score will be lower.

- Genre granularity: Genre labels are coarse. Horror covers slasher films, psychological thrillers, and supernatural horror equally. The keyword and overview signals partially compensate for this but do not fully resolve it.
- Language bias: The dataset is English-centric. Non-English films such as Parasite, Amelie, and Seven Samurai may have sparser keyword and overview representations, slightly underperforming their true similarity to thematically related English-language films.

8.4 Recommendations for Future Work

- Semantic embeddings: Replace TF-IDF with sentence-transformer embeddings for the overview signal. Semantic embeddings capture that robbery and heist are the same concept — TF-IDF does not.
- Approximate Nearest Neighbours: For production deployment with millions of movies, replace exact cosine similarity with FAISS for sub-millisecond query times at any scale.
- True hybrid system: If user rating data becomes available, integrate a collaborative filtering component to produce a genuine content-plus-behaviour hybrid, which would achieve higher recommendation quality than either approach alone.
- Dynamic weight optimisation: Use a labelled validation set of human-rated recommendation quality to optimise the five signal weights via grid search or Bayesian optimisation rather than setting them manually.

9. Deployment

The system is deployed as a Flask web application that loads the four PKL artefacts at startup and serves recommendations via a REST API consumed by a browser-based frontend.

9.1 Architecture Overview

- Backend: Flask (Python) — loads combined_matrix.pkl, movie_meta.pkl, title_index.pkl, and titles_list.pkl at startup. Exposes /api/search for autocomplete and /api/recommend for inference.
- Frontend: Single-page HTML/CSS/JavaScript application with live autocomplete search, animated recommendation cards, and TMDB poster images loaded via CDN.
- Inference: One sparse dot product per query — typically under 50 ms for 50,000 movies on any modern CPU.

9.2 Download PKLs from Colab

```
from google.colab import files
files.download('/content/models/combined_matrix.pkl')
files.download('/content/models/movie_meta.pkl')
files.download('/content/models/title_index.pkl')
files.download('/content/models/titles_list.pkl')
```

9.3 Run Locally

```
pip install flask pandas numpy scikit-learn scipy
# Place the 4 pkl files in the models/ folder
python app.py
# Open http://localhost:5000
```

9.4 Repository Structure

```
content-movie-recommender/
  models/
    combined_matrix.pkl
    movie_meta.pkl
    title_index.pkl
    titles_list.pkl
  templates/
    index.html
  app.py
  requirements.txt
  colab_training.py
  README.md
```

10. Documentation & Presentation

10.1 Submission Checklist

Deliverable	Description	Status
Project Report (DOCX/PDF)	This document — all 10 sections updated for new architecture	Complete
Colab Training Script	Single-cell Colab script: mount Drive, unzip, train, save PKLs	Complete
combined_matrix.pkl	Sparse 50K × 13K weighted feature matrix (150–200 MB)	Complete
movie_meta.pkl	Movie metadata: title, genres, directors, cast, poster, etc.	Complete
title_index.pkl	Dictionary: movie title →row index in combined matrix	Complete
titles_list.pkl	Ordered list of movie titles (index → title)	Complete
app.py	Flask backend with /api/search and /api/recommend endpoints	Complete
templates/index.html	Dark cinema frontend with autocomplete and poster cards	Complete
requirements.txt	flask, pandas, NumPy, scikit-learn, SciPy	Complete

10.2 Presentation Outline (12 Slides)

#	Title	Key Points
1	Title Slide	Project name, COMP 413, team members, Kabarak University, 2026
2	Problem Statement	Movie discovery problem, business impact, why metadata-only approach
3	Dataset Overview	367K movies, 50K working set, key columns, TMDB+IMDB source
4	Data Cleaning	Released filter, vote threshold, missing values, director triple-weighting
5	EDA	Vote distribution, genre coverage, keyword density, director clusters
6	Memory Problem	Why N×N matrix crashes Colab; sparse vector architecture as solution
7	5-Signal Architecture	Diagram: genre + keywords + people + overview + numeric →combined
8	Feature Engineering	TF-IDF variants, log-scaling, L2-normalise, weighted hstack
9	Inference Architecture	Sparse dot product query, argpartition, RAM comparison table
10	Evaluation	Ablation comparison table, Inception example, signal contributions
11	Live Demo	Run Flask app, search Inception, show poster cards with scores
12	Conclusion & References	Summary, limitations, future work, Q&A

10.3 Conclusion

This project delivers a fully functional content-based movie recommendation system that fuses five complementary signals — genre, plot keywords, creative personnel, overview text, and numeric quality metrics — into a single weighted sparse feature matrix. The central architectural innovation is storing normalised feature vectors rather than a pre-computed pairwise similarity matrix, reducing peak training RAM from an intractable 540 GB (for 367K movies) to a manageable 2–3 GB, while producing mathematically identical similarity scores at query time.

The system demonstrates that high-quality recommendations do not require user behaviour data. Rich movie metadata alone, properly engineered and combined, produces intuitive and defensible recommendations: films by the same director cluster together, thematically specific keyword profiles pull together films with similar narratives, and the numeric signal ensures that wildly different audience reception profiles push films apart even when their content overlaps.

The Flask web application makes the system accessible through a browser with live autocomplete, TMDB poster images, and animated recommendation cards — a production-ready deployment of an academically rigorous content-based recommendation pipeline.

References

- Dataset: TMDB + IMDB Movies Dataset — Kaggle
- Scikit-learn Documentation: https://scikit-learn.org
- Flask Documentation: https://flask.palletsprojects.com
- Ricci, F., Rokach, L., & Shapira, B. (2011). Recommender Systems Handbook. Springer.
- Burke, R. (2002). Hybrid Recommender Systems: Survey and Experiments. User Modeling and User-Adapted Interaction, 12(4), 331–370.
- Johnson, J. et al. (2019). Billion-scale similarity search with GPUs. IEEE Transactions on Big Data. [FAISS reference]
- Netflix Tech Blog — System Architectures for Personalization: https://netflixtechblog.com